INSTITUTO TÉCNICO DE EDUCACIÓN COMERCIAL AMERICANO
Programación de Dispositivos Móviles II

TiendaCerca

Aplicación móvil para la búsqueda de tiendas cercanas y sus ofertas

Documentación final del proyecto

Presentado por:
Roberto

Docente: Ing. Alexander Pabon Pabon
20 de octubre de 2026

Índice

1. Introducción

2. Planteamiento del problema

3. Justificación

4. Objetivos

5. Alcance del proyecto

6. Tecnologías utilizadas

7. Requisitos del sistema

8. Historias de usuario

9. Tipos de usuario y roles

10. Arquitectura del sistema

11. Modelo de datos

12. Diseño de interfaz

13. Cronograma del proyecto

14. Desarrollo del proyecto, semana a semana

15. Pruebas realizadas

16. Registro y corrección de errores

17. Resultados obtenidos

18. Conclusiones

19. Recomendaciones y trabajo futuro

20. Referencias

1. Introducción

TiendaCerca es una aplicación móvil desarrollada en Flutter para la materia Programación de Dispositivos Móviles II, del Instituto Técnico de Educación Comercial Americano. El proyecto nació como “FerreCerca”, enfocado solo en ferreterías, y se amplió —por pedido del docente, tras la revisión de la primera entrega— para cubrir cualquier tipo de comercio local: ferreterías, abarrotes, ropa, farmacias, electrónica y restaurantes, entre otros.

A lo largo de ocho semanas de desarrollo incremental, siguiendo la bitácora de seguimiento de la materia, la aplicación pasó de ser una idea en papel a una app funcional con geolocalización real, persistencia local en SQLite, autenticación de usuarios con usuario y contraseña, y tres tipos de cuenta (Cliente, Dueño de tienda y Administrador) con navegación diferenciada. Este documento reúne, de principio a fin, el planteamiento, el diseño, el desarrollo, las pruebas y los resultados de todo el proyecto.

---

2. Planteamiento del problema

En ciudades como La Paz, gran parte del comercio es local y de barrio: ferreterías, tiendas de abarrotes, boutiques de ropa, farmacias, tiendas de electrónica y restaurantes pequeños. Para un cliente que no conoce la zona, o que simplemente quiere comparar antes de salir de casa, es difícil saber qué tiendas cercanas existen, qué ofertas tienen vigentes en ese momento y cómo contactarlas o llegar hasta ellas, salvo que ya las conozca de antes.

Del otro lado, un comerciante pequeño rara vez cuenta con un canal digital propio —una app o una página— para anunciar sus promociones. Suele depender del boca a boca o de redes sociales genéricas, donde su negocio se mezcla con el de cualquier otro rubro y no es fácil de encontrar por cercanía ni por categoría de producto.

3. Justificación

El uso de teléfonos inteligentes con GPS es ya prácticamente universal, y herramientas como Flutter permiten construir una aplicación multiplataforma a partir de un único código base, usando OpenStreetMap —gratuito, sin necesitar una llave de API de pago— para mostrar un mapa real. Esto hace viable, incluso para un proyecto académico sin presupuesto, construir algo cercano a un producto real.

Desde el punto de vista académico, TiendaCerca permite aplicar de forma práctica e incremental los contenidos de la materia: geolocalización real, persistencia local con SQLite, gestión de estado con Provider, formularios y validaciones, autenticación de usuarios, roles diferenciados y pruebas automatizadas, todo dentro de un mismo proyecto que fue creciendo semana a semana en vez de construirse de una sola vez.

Desde el punto de vista social, el proyecto —aunque sea a nivel de práctica, sin backend ni usuarios reales todavía— le da una primera forma concreta a una herramienta que podría ayudar a visibilizar el comercio de barrio frente a las grandes cadenas, y a que un comerciante pequeño publique sus ofertas sin depender de terceros.

---

4. Objetivos

4.1 Objetivo general

Desarrollar una aplicación móvil, usando Flutter, que permita a los usuarios encontrar tiendas cercanas por categoría mediante geolocalización real, consultar sus ofertas vigentes, y que permita a cada comerciante registrar y administrar su propia tienda y sus propias promociones, con tres tipos de usuario diferenciados dentro de la misma app.

4.2 Objetivos específicos

- Implementar la búsqueda y el filtrado de tiendas cercanas por categoría, usando la ubicación real del dispositivo (geolocator) y un mapa interactivo (flutter_map sobre OpenStreetMap).
- Diseñar y construir una base de datos local (SQLite) que persista cuentas, tiendas, favoritos y ofertas publicadas entre sesiones, sin depender de un servidor remoto.
- Implementar un sistema de autenticación real: registro e inicio de sesión con usuario y contraseña, con la contraseña siempre guardada como hash, nunca en texto plano.
- Construir tres tipos de usuario con navegación diferenciada —Cliente, Dueño de tienda y Administrador— cada uno viendo y pudiendo hacer solo lo que le corresponde.
- Permitir que cada Dueño de tienda registre su propia tienda (nombre, categoría, ubicación real elegida en el mapa y una foto) y publique, edite, pause o elimine sus ofertas, también con foto.
- Dar al Administrador una vista de supervisión de toda la plataforma, con la capacidad de moderar —pausar o eliminar— cualquier oferta publicada, no solo las de su propia tienda.
- Verificar el funcionamiento de la aplicación mediante pruebas automatizadas (flutter_test) y pruebas manuales en un celular real y en emuladores.

5. Alcance del proyecto

Incluido en esta entrega:

- Aplicación Android, compilada y probada en un celular físico real y en varios emuladores (Android 14 y 16).
- Catálogo de ejemplo de 10 tiendas de rubros variados (de solo lectura), pensado como reemplazo futuro por un backend/API real.
- Registro de tiendas reales por los propios Dueños de tienda, con nombre, categoría, dirección, ubicación real en el mapa y foto.
- Favoritos, y publicación, edición, pausa y eliminación de ofertas con foto.
- Autenticación real (registro e inicio de sesión) y tres roles de usuario.
- Pruebas automatizadas (unitarias y de interfaz) y manuales.

Fuera de alcance de esta entrega:

- Un backend remoto o API propia: todos los datos reales viven en la base de datos SQLite local del dispositivo, sin sincronización entre dispositivos.
- Pasarela de pagos y notificaciones push.
- Compilación y prueba en iOS o en web (no verificada en esta entrega).
- Edición de la tienda después de registrarla (hoy el nombre, la categoría, la ubicación y la foto se fijan al crear la cuenta).

---

6. Tecnologías utilizadas

Tecnología	Uso dentro del proyecto
Flutter / Dart	Framework multiplataforma; todo el código de la app.
provider	Gestión de estado (AppState extends ChangeNotifier) compartido entre pantallas.
sqflite	Base de datos SQLite local en el dispositivo (cuentas, tiendas, favoritos, ofertas, sesión).
sqflite_common_ffi	Permite correr SQLite real dentro de las pruebas automatizadas (flutter test), sin emulador.
geolocator	Ubicación real del dispositivo (permisos, posición, distancia a cada tienda).
flutter_map + latlong2	Mapa interactivo con OpenStreetMap, sin necesitar una llave de API.
image_picker	Fotos reales de ofertas y de tiendas, desde cámara o galería.
crypto	Hash SHA-256 de las contraseñas antes de guardarlas.
url_launcher	Abre “Cómo llegar” (Maps) y “Llamar” (discador) en apps externas.
google_fonts	Tipografía del diseño (Space Grotesk).
flutter_test	Pruebas automatizadas unitarias y de interfaz.

7. Requisitos del sistema

7.1 Requisitos funcionales

1. El sistema debe permitir registrar una cuenta nueva (nombre, usuario, contraseña) eligiendo el rol Cliente o Dueño de tienda.
2. El sistema debe permitir iniciar sesión con usuario y contraseña, y mantener la sesión entre reinicios de la app.
3. El sistema debe permitir buscar y filtrar tiendas por categoría y por texto (nombre o zona).

4. El sistema debe mostrar el detalle de una tienda: ubicación en el mapa, distancia, horario, teléfono, dirección y sus ofertas vigentes.
5. El sistema debe permitir marcar y quitar una oferta como favorita.
6. El sistema debe permitir que un Dueño de tienda registre su propia tienda, con nombre, categoría, dirección, ubicación real (elegida tocando un mapa) y una foto opcional.
7. El sistema debe permitir que un Dueño de tienda publique, edite, pause o elimine las ofertas de su propia tienda, con foto incluida.
8. El sistema debe permitir que un Administrador vea todas las tiendas de la plataforma y pause o elimine cualquier oferta publicada, no solo las de su propia tienda.
9. El sistema debe permitir cerrar sesión desde la pestaña Perfil.

7.2 Requisitos no funcionales

- Los datos del usuario (sesión, favoritos, tiendas y ofertas propias) deben persistir localmente, incluso sin conexión a internet, ya que no dependen de un servidor remoto.
- Si la base de datos local no está disponible, la aplicación no debe caerse: debe seguir funcionando con los datos en memoria y avisar al usuario.
- Las contraseñas nunca deben guardarse en texto plano.
- La interfaz debe adaptarse a distintos tamaños de pantalla (uso de listas desplazables en vez de layouts de tamaño fijo).
- La aplicación debe funcionar igual en distintas versiones de Android (probado en Android 14 y 16) sin cambios de código.
- Las operaciones comunes (buscar, favoritos, publicar) deben sentirse instantáneas: la interfaz se actualiza en memoria primero y la base de datos se escribe en segundo plano.

---

8. Historias de usuario

- Como Cliente, quiero buscar tiendas cercanas por categoría, para encontrar rápido dónde comprar algo que necesito.

- Como Cliente, quiero ver la distancia real y la ubicación de una tienda en el mapa, para decidir si me conviene ir.
- Como Cliente, quiero guardar ofertas como favoritas, para encontrarlas de nuevo sin tener que buscarlas otra vez.
- Como Dueño de tienda, quiero registrar mi propia tienda con su ubicación real y una foto, para que los clientes me encuentren en el mapa y reconozcan mi negocio.
- Como Dueño de tienda, quiero publicar ofertas con foto, precio y descripción, para atraer clientes.
- Como Dueño de tienda, quiero poder pausar o eliminar una oferta, para dejar de mostrarla cuando ya no está disponible.
- Como Administrador, quiero ver todas las tiendas de la plataforma, para tener una vista general del negocio.
- Como Administrador, quiero poder pausar o eliminar cualquier oferta publicada, para moderar contenido inapropiado aunque no sea de mi propia tienda.
- Como usuario de cualquier rol, quiero que mi sesión se recuerde entre reinicios, para no tener que iniciar sesión cada vez que abro la app.

9. Tipos de usuario y roles

TiendaCerca reconoce tres tipos de usuario distintos, que ven y pueden hacer cosas diferentes dentro de la misma aplicación: no tendría sentido que cualquier persona que abre la app viera exactamente las mismas pantallas, sin importar si busca tiendas, administra la suya propia, o modera la plataforma.

Rol	Qué puede hacer	Pantallas que ve
Cliente	Busca tiendas cercanas por categoría, ve sus ofertas y guarda favoritos.	Buscar, Favoritos, Perfil
Dueño de tienda	Todo lo del Cliente, más registrar su propia tienda (ubicación y foto) y publicar, editar, pausar o eliminar sus ofertas.	Buscar, Favoritos, Mi tienda, Perfil
Administrador	Ve el catálogo completo de tiendas de la plataforma y puede pausar o eliminar cualquier	Buscar, Admin, Perfil

	oferta publicada, no solo las propias (moderación).	

El registro y el inicio de sesión son reales: cada cuenta (usuario, nombre, contraseña con hash y rol) se guarda en la base de datos SQLite local. El rol Administrador no se registra desde la app —ya existe una cuenta sembrada por defecto (usuario admin) que entra por el mismo formulario que cualquier otra cuenta, sin exponer un botón de registro de administrador. La sesión se recuerda entre reinicios hasta que se cierre sesión desde la pestaña Perfil.

10. Arquitectura del sistema

TiendaCerca está organizada en capas, de forma que cada parte del código tiene una sola responsabilidad y no necesita saber cómo funcionan las demás por dentro. Esto es lo que permitió, por ejemplo, agregar el registro de tiendas reales (semana 8) sin tener que tocar las pantallas de búsqueda: solo cambió de dónde viene el dato, no cómo se muestra.

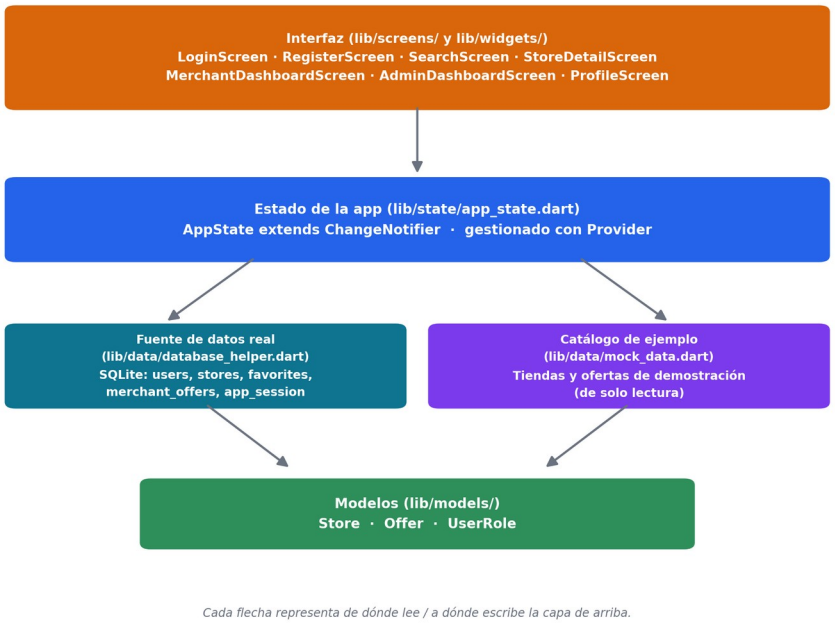


Figura 1. Arquitectura por capas de TiendaCerca.

- Interfaz (lib/screens/ y lib/widgets/): las pantallas y widgets reutilizables (tarjetas, chips, mapa). No acceden directamente a la base de datos; solo leen del estado y disparan acciones.
- Estado de la app (lib/state/app_state.dart): una sola clase AppState (ChangeNotifier) gestionada con el paquete provider, que concentra la sesión, los filtros de búsqueda, los favoritos y las operaciones sobre tiendas y ofertas.
- Fuente de datos: DatabaseHelper (SQLite real, lib/data/database_helper.dart) para todo lo que el usuario guarda de verdad, y MockData (lib/data/mock_data.dart) para el catálogo de tiendas de ejemplo, que conviven lado a lado y se combinan en AppState.allStores.
- Modelos (lib/models/): Store, Offer y UserRole — clases de datos simples, sin lógica de negocio, usadas por las tres capas de arriba.

11. Modelo de datos

Toda la información que el usuario genera dentro de la app (su cuenta, su tienda, sus ofertas, sus favoritos) se guarda localmente en una base de datos SQLite (tiendacerca.db), manejada por el paquete sqflite. El catálogo general de tiendas de ejemplo no vive en esta base: es de solo lectura, pensado para reemplazarse después por un backend real.

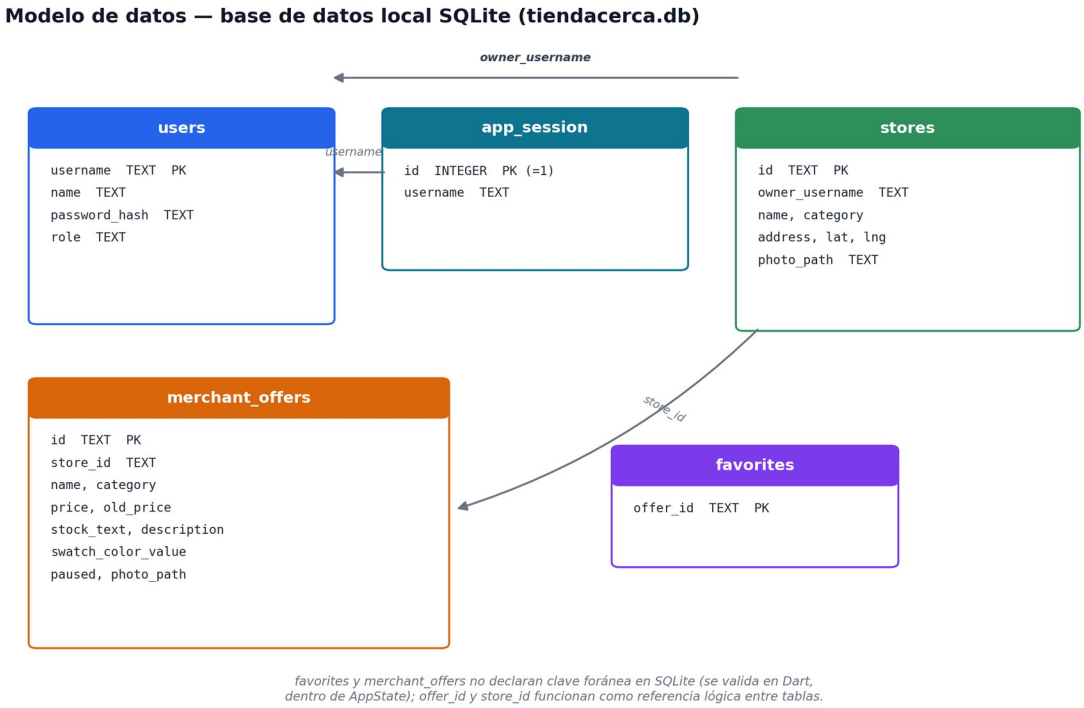


Figura 2. Modelo de datos de la base SQLite local (tiendacerca.db).

- users: una fila por cuenta registrada (usuario único, nombre, contraseña con hash SHA-256 y rol). Incluye la cuenta Administrador sembrada por defecto.
- app_session: una sola fila que recuerda qué cuenta sigue con la sesión abierta entre reinicios de la app.
- stores: la tienda real de cada cuenta Dueño de tienda (id determinístico store-<usuario>, nombre, categoría, dirección, latitud/longitud y foto opcional).
- merchant_offers: las ofertas publicadas, ligadas a una tienda por store_id; cada una puede pausarse o eliminarse sin afectar a las demás.
- favorites: los id de oferta que el usuario marcó como favorita.

12. Diseño de interfaz

Los colores, tipografías y estilos de botones, tarjetas y campos están centralizados en lib/theme/app_theme.dart, así que cambiar la paleta de toda la app es cuestión de editar un solo archivo en vez de buscar estilos repetidos pantalla por pantalla. El diseño es responsivo: las listas usan scroll en vez de tamaños fijos, para adaptarse a distintos tamaños de pantalla.

- Iniciar sesión / Crear cuenta: formulario de usuario y contraseña, o registro con nombre, usuario, contraseña y el rol (Cliente o Dueño de tienda); si el rol es Dueño de tienda, aparecen además los campos de su propia tienda (nombre, categoría, dirección, mapa para elegir ubicación y foto opcional).
- Buscar: mapa real (OpenStreetMap) con las tiendas cercanas, más una lista filtrable por categoría y texto, ordenada por distancia real (GPS).
- Detalle de tienda: foto o color de la tienda, calificación, distancia, botones "Cómo llegar" y "Llamar", y sus ofertas vigentes.
- Detalle de oferta: foto, precio, descuento, descripción y el botón de favorito.
- Favoritos: lista de las ofertas que el usuario marcó, con acceso directo a su detalle.
- Mi tienda (rol Dueño de tienda): datos de la propia tienda, estadísticas y la lista de sus ofertas, con publicar/editar/pausar/eliminar.
- Publicar/editar oferta: formulario con foto (cámara o galería), nombre, precio, precio anterior, descripción y existencias.
- Admin (rol Administrador): catálogo completo de tiendas de la plataforma y moderación de cualquier oferta publicada.
- Perfil: rol actual, nombre de usuario y botón de cerrar sesión, visible para los tres roles.

13. Cronograma del proyecto

El proyecto se organizó en 8 semanas, siguiendo los indicadores de la bitácora de seguimiento de la materia, desde la idea inicial hasta el cierre y la documentación final.

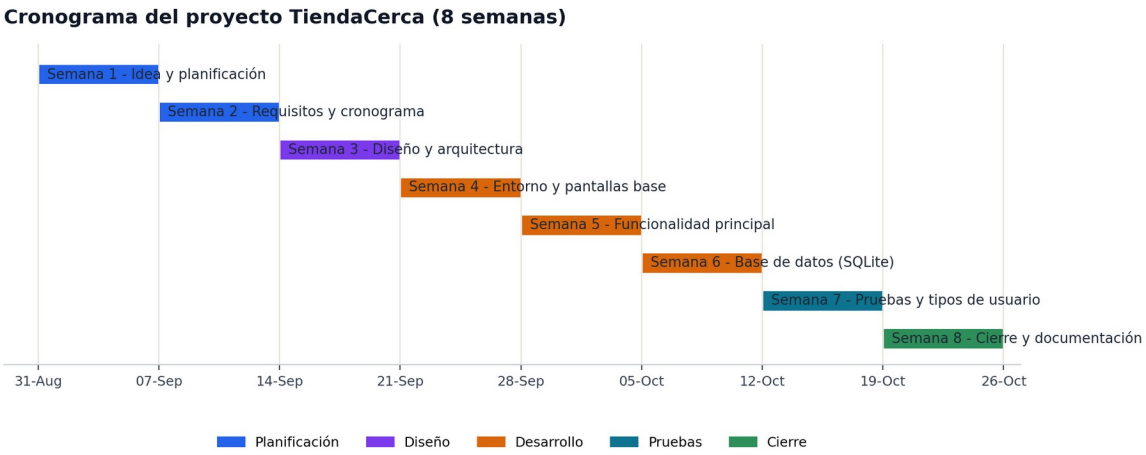


Figura 3. Cronograma de 8 semanas del proyecto TiendaCerca (31 de agosto – 25 de octubre de 2026).

Semana	Fechas	Fase	Entregable principal
1	31 ago – 6 sep	Planificación	Problema, usuario, solución, objetivos y alcance
2	7 – 13 sep	Planificación	Requisitos, historias de usuario y cronograma
3	14 – 20 sep	Diseño	Wireframes, navegación, arquitectura y modelo de datos
4	21 – 27 sep	Desarrollo	Entorno configurado, estructura del proyecto y pantallas base
5	28 sep – 4 oct	Desarrollo	Funcionalidad principal, formularios, validaciones y manejo de estado
6	5 – 11 oct	Desarrollo	Base de datos SQLite real (favoritos y ofertas propias)
7	12 – 18 oct	Pruebas	Tipos de usuario, pruebas automatizadas y corrección de errores
8	19 – 25 oct	Cierre	Login real, registro de tienda propia, revisión final y documentación

14. Desarrollo del proyecto, semana a semana

Semana 1 — Idea y planificación

Se definió el problema, el usuario objetivo, la solución y los objetivos del proyecto, originalmente enfocado solo en ferreterías cercanas ("FerreCerca"). El docente pidió que el alcance fuera más amplio y cubriera cualquier tipo de tienda, no solo ferreterías.

Semana 2 — Requisitos y cronograma

Se documentaron los requisitos funcionales y no funcionales, las historias de usuario, la priorización de tareas y un primer cronograma. Se decidió renombrar el proyecto a "TiendaCerca" para reflejar el alcance ampliado a todo tipo de comercio.

Semana 3 — Diseño y arquitectura

Se diseñaron los wireframes de las pantallas principales, el diagrama de navegación y la arquitectura por capas del proyecto, ya con el catálogo de ejemplo ampliado a rubros variados (ferretería, abarrotes, ropa, farmacia, electrónica, restaurantes).

Semana 4 — Entorno y pantallas base

Se configuró el proyecto Flutter, se definió la estructura de carpetas (models/, data/, state/, screens/, widgets/) y se construyeron las pantallas principales con navegación entre ellas. Se probó la app corriendo en un celular Android real.

Semana 5 — Funcionalidad principal

Se implementó la búsqueda y filtrado de tiendas, el manejo de estado centralizado con provider, formularios con validación y la base del manejo de errores (por ejemplo, un fallo de geolocalización no debía tumbar la app).

Semana 6 — Base de datos (SQLite)

Se conectó la app a una base de datos SQLite local real (paquete sqflite) para favoritos y ofertas propias del comerciante, con lectura al iniciar la app y escritura al publicar, pausar o marcar favoritos. Se corrigió un bug de choque de nombres que impedía compilar la app y se agregaron los permisos de ubicación y cámara que faltaban. También se integró image_picker para fotos reales de las ofertas.

Semana 7 — Pruebas y tipos de usuario

Se implementaron los tres tipos de usuario (Cliente, Dueño de tienda, Administrador) con una pantalla de inicio de sesión, un panel de Administrador para moderación, y se agregó edición y eliminación de ofertas (antes solo se podían pausar). Se escribieron pruebas automatizadas con flutter_test (modelos y navegación por rol) y se corrigieron advertencias del analizador por APIs obsoletas.

Semana 8 — Cierre y documentación

Se reemplazó el inicio de sesión por código especial por un login real con usuario y contraseña (guardados con hash en SQLite), ya que un/a evaluador/a señaló que el código especial no era un login real. Se agregó que cada Dueño de tienda registre su propia tienda real, con ubicación elegida en el mapa y una foto, en vez de compartir todos la misma tienda de ejemplo. Se hizo una revisión final de código (manejo seguro de ids inexistentes, nombre de la app y configuración del mapa) y se escribió esta documentación final, con diagrama de Gantt, diagrama de arquitectura y modelo de datos.

15. Pruebas realizadas

15.1 Pruebas unitarias

test/models_test.dart prueba la lógica de los modelos de datos sin necesidad de levantar pantallas: el cálculo de precio y descuento de Offer, y los valores de UserRole.

15.2 Pruebas de interfaz (widget tests)

test/role_navigation_test.dart usa sqflite_common_ffi para correr una base de datos SQLite real dentro de flutter test (sin emulador ni celular) y cubre, de punta a punta:

- Que sin sesión iniciada se muestre el login con usuario y contraseña.
- Registrarse como Cliente: solo ve las pestañas Buscar, Favoritos y Perfil.
- Registrarse como Dueño de tienda: aparecen los campos de su propia tienda (nombre, categoría, dirección, mapa y foto), la cuenta ve la pestaña "Mi tienda" sin ofertas propias todavía, y puede publicar una oferta que luego aparece en el panel.
- Iniciar sesión con la cuenta Administrador sembrada: entra al panel de Admin y ve tanto el catálogo de ejemplo como la tienda y la oferta reales registradas por la cuenta Dueño de tienda de la prueba anterior (moderación entre cuentas distintas).
- Una contraseña incorrecta muestra un error y no deja entrar.
- Cerrar sesión desde el Perfil regresa a la pantalla de login.

15.3 Pruebas manuales y multiplataforma

Además de las pruebas automatizadas, la app se probó manualmente en un celular Android físico y en varios emuladores (Android 14 y 16): flujo completo de registro, búsqueda con mapa y GPS real, publicar/editar/pausar/eliminar una oferta con foto desde cámara y desde galería, y moderación desde el panel de Administrador. No se ha probado todavía en iOS, web ni escritorio, aunque en teoría Flutter lo permite sin cambios de código.

15.4 Pruebas de regresión

Cada semana se verificó que lo ya construido siguiera funcionando igual tras agregar funcionalidad nueva: la búsqueda y el filtro por categoría, marcar/quitar favoritos, publicar/editar/pausar/eliminar una oferta, las fotos reales y el recordar la sesión entre reinicios se revisaron de nuevo al agregar login real (semana 8) y registro de tienda propia (semana 8), sin que se rompiera ninguno.

16. Registro y corrección de errores

Historial real de errores encontrados durante el desarrollo y cómo se corrigieron:

Semana	Error encontrado	Causa	Corrección
4–5	La app no compilaba ("algo salió mal")	Choque de nombre entre una clase propia y ColorSwatch de Flutter; faltaban permisos de ubicación	Clase renombrada a SwatchBox; permisos agregados en AndroidManifest.xml
6	Las ofertas publicadas no aparecían al verlas como comprador	Las pantallas de comprador leían el catálogo fijo de MockData, no los datos reales guardados en SQLite	Se agregó AppState.offersForStore()/findOffer() como única fuente de verdad
6	El botón de agregar foto no hacía nada real (solo cambiaba de color)	Era un simple placeholder visual, sin selección de imagen	Se integró image_picker (cámara/galería) con permiso de cámara
6	Un fallo de geolocalización podía tumbar la app	location_service.dart solo envolvía la última línea del método en try/catch, no todo el método	Todo el método getCurrentLocation() quedó dentro de un único try/catch
7	No se podía editar ni eliminar una oferta publicada (solo pausar)	La funcionalidad de editar/eliminar no estaba implementada	Se agregó edición (formulario precargado) y eliminación (con diálogo de confirmación)
7	Advertencias del analizador por APIs obsoletas	Cambios en las APIs de Flutter entre versiones del SDK	Reemplazadas withOpacity por withValues(alpha:) y value: por initialValue:
8	El inicio de sesión con código especial no era un login real	Faltaba un formulario real de usuario y contraseña con cuentas guardadas	Se agregó la tabla users (contraseña con hash) y un formulario real de usuario/contraseña
8	StoreDetailScreen podía reventar con un StateError si el id de la tienda no existía	Buscaba la tienda con firstWhere sin manejar el caso de no encontrarla	Se reemplazó por una búsqueda segura que muestra "Tienda no disponible" en vez de caerse
8	El nombre de la app en el launcher del teléfono seguía diciendo	AndroidManifest.xml no se había actualizado al renombrar	Se corrigió el android:label en AndroidManifest.xml

	"ferrecerca_app"	el proyecto a TiendaCerca	

17. Resultados obtenidos

Al cierre de la semana 8, TiendaCerca cumple los requisitos planteados desde la semana 2: los tres tipos de usuario funcionan con un login real, la búsqueda por categoría y distancia usa GPS real, los Dueños de tienda registran su propia tienda (ubicación y foto) y publican ofertas con foto, y el Administrador puede moderar cualquier oferta de la plataforma. Todo lo que el usuario realmente guarda (cuenta, sesión, tienda propia, ofertas y favoritos) persiste en una base de datos SQLite local, y la app no se cae ante datos faltantes o fallos del GPS gracias al manejo de errores agregado en distintas semanas.

El catálogo general de tiendas de ejemplo (mock_data.dart) sigue siendo de demostración a propósito, como punto de partida para conectar un backend real más adelante, pero convive sin conflictos con las tiendas reales registradas por los Dueños de tienda.

18. Conclusiones

Desarrollar TiendaCerca a lo largo de 8 semanas permitió aplicar de forma práctica los temas de la materia: manejo de estado con Provider, persistencia local con SQLite, geolocalización real, permisos de dispositivo (ubicación y cámara), diferenciación de roles de usuario con autenticación real, y pruebas automatizadas de interfaz. El proyecto también mostró la importancia de mantener una arquitectura por capas: varios cambios grandes (agregar login real, o que cada Dueño de tienda tenga su propia tienda) fueron posibles sin reescribir las pantallas, porque la lógica de datos y de estado estaba separada de la interfaz desde el principio.

Igual de importante fue la retroalimentación recibida en el camino (por ejemplo, que el código especial "no era un login real"): corregirlo a tiempo, en vez de dejarlo para el final, evitó acumular una deuda técnica mayor antes de la entrega final.

19. Recomendaciones y trabajo futuro

- Conectar un backend remoto real (Firebase, Supabase, o una API propia) para el catálogo de tiendas — hoy solo los datos propios del usuario están persistidos localmente.
- Agregar notificaciones push cuando una tienda que el usuario sigue publica una oferta nueva.
- Dejar que un Dueño de tienda edite su tienda después de registrarla (hoy el nombre, la ubicación y la foto se fijan al crear la cuenta).
- Ampliar las pruebas automatizadas para cubrir también los flujos de editar y eliminar una oferta de punta a punta, no solo publicarla.
- Probar y validar la app en iOS, web y escritorio, plataformas que Flutter soporta en teoría pero que todavía no se han verificado en este proyecto.

---

20. Referencias

Flutter. (2026). Flutter documentation. Google. https://docs.flutter.dev

geolocator (paquete Dart). (2026). pub.dev. https://pub.dev/packages/geolocator

image_picker (paquete Dart). (2026). pub.dev. https://pub.dev/packages/image_picker

flutter_map (paquete Dart). (2026). pub.dev. https://pub.dev/packages/flutter_map

OpenStreetMap contributors. (2026). OpenStreetMap. https://www.openstreetmap.org

provider (paquete Dart). (2026). pub.dev. https://pub.dev/packages/provider

crypto (paquete Dart). (2026). pub.dev. https://pub.dev/packages/crypto

sqflite (paquete Dart). (2026). pub.dev. https://pub.dev/packages/sqflite